

Project Setup Instructions

To set up and run this project locally, follow these steps:

Prerequisites:

- **Python (3.9+ recommended):** Download and install from python.org.
- **pip:** Python's package installer (usually comes with Python).
- **Docker:** Download and install Docker Desktop from docker.com.
- **(Optional) ngrok:** For exposing local services (like MLflow UI) to the internet. Install via `pip install pyngrok` and obtain an authentication token from dashboard.ngrok.com/auth.

Step 1: Clone the Repository (or Download Notebook)

If this project is part of a Git repository, clone it:

```
git clone https://github.com/2025paml563-coder/mlminiproject.git  
cd mlminiproject
```

If you have a .ipynb file, simply save it to your local machine.

Step 2: Prepare the Data

1. **Download NY_Trip_Raw_Data.csv:** This dataset is the raw input for the project. Ensure it's placed in the correct location (e.g., [/content/sample_data/NY_Trip_Raw_Data.csv](#) if replicating the Colab environment, or adjust the path in the notebook).
 - The notebook assumes the raw data is at [/content/sample_data/NY_Trip_Raw_Data.csv](#) within the Colab environment.

Step 3: Set up the Python Environment

It's highly recommended to use a virtual environment.

```
python -m venv venv  
source venv/bin/activate # On Windows, use `venv\Scripts\activate`
```

Install the required Python packages:

```
pip install pandas numpy scikit-learn matplotlib seaborn mlflow pyngrok gunicorn
```

Step 4: Run the Jupyter Notebook (or execute cells sequentially)

Open the notebook (.ipynb file) in your preferred environment (e.g., Jupyter Lab, VS Code, or Google Colab if you are running it there):

1. **Open the Notebook:** If using Jupyter:

jupyter notebook `` Then navigate to and open the notebook file.

2. **Execute Cells Sequentially:** Run all cells in the notebook from top to bottom. * Pay attention to outputs and any warnings. * The notebook will perform data loading, preprocessing, model training, MLflow logging, data drift simulation, and generate Dockerfiles.

Step 5: Access the MLflow UI (Optional, for Experiment Tracking)

The notebook includes a cell (6sAiHhhMnld-) to start the MLflow UI and expose it via ngrok. Execute this cell.

1. **Locate the MLflow UI Cell:** Find the cell that starts mlflow ui.
2. **Run the Cell:** Execute this cell to start the MLflow UI .
3. **Open in Browser:** Copy the provided URL and paste it into your web browser to view the MLflow UI, where you can inspect all logged runs, metrics, parameters, and models.

Step 6: Serve the Model using Docker (Self-Contained Image)

The notebook generates a self-contained Dockerfile and requirements.txt (and downloads model artifacts into linear_regression_model_artifacts). You'll need to transfer these files to your local machine.

1. **Download Generated Files:** After running the relevant cells (specifically LGWHJ3wLtEam), download the following from your Colab (or execution) environment to a new local directory:
 - Dockerfile
 - requirements.txt
 - The entire directory: linear_regression_model_artifacts
 - Alternatively we can also convert this to a .pkl file and use it in docker.
2. **Navigate to Directory:** Open your terminal and change to the local directory where you downloaded these files.
3. **Build the Docker Image:**
- 4.

`docker build -t my-taxi-duration-predictor . ``` (Replace my-taxi-duration-predictor with your desired image name).

5: Run the Docker Container:

`docker run -p 5001:5001 my-taxi-duration-predictor ``` This will start a web server inside the Docker container, serving your model on <http://localhost:5001/predict>

6: Test the Model Endpoint

Once the Docker container is running, you can send prediction requests.

1. **Prepare JSON Payload:** Use the example JSON payload provided in the notebook (e.g., in cell 4bd241de). Ensure the columns and data match the model's expected input schema.
2.

```
{
  "dataframe_split": {
    "columns": [
      "passenger_count", "pickup_longitude", "pickup_latitude", "dropoff_longitude",
      "dropoff_latitude", "pickup_hour", "pickup_dayofweek", "is_weekend",
      "distance_km", "weather_freezing", "weather_rainy", "weather_snowy",
      "weather_sunny", "weather_windy", "store_and_fwd_flag_Y", "vendor_id_2"
    ],
    "data": [
      [1, -73.987654, 40.756789, -
73.971234, 40.768901, 18, 2, 0, 2.5, 0, 0, 0, 1, 0, 0, 1]
    ]
  }
}
```
3. **Send Request using curl:** Open a new terminal window (keep the Docker container running) and execute:
- 4.

```
curl -X POST -H "Content-Type: application/json"
--data '{"dataframe_split": {"columns": ["passenger_count", "pickup_longitude",
"pickup_latitude", "dropoff_longitude", "dropoff_latitude", "pickup_hour",
"pickup_dayofweek", "is_weekend", "distance_km", "weather_freezing",
"weather_rainy", "weather_snowy", "weather_sunny", "weather_windy",
"store_and_fwd_flag_Y", "vendor_id_2"], "data": [[1, -73.987654, 40.756789, -
73.971234, 40.768901, 18, 2, 0, 2.5, 0, 0, 0, 1, 0, 0, 1]]}}'
```

<http://localhost:5000/invocations> ``` You should receive a JSON response containing the predicted trip duration.

This completes the setup and demonstration of the project's MLOps pipeline components.